Contents

Reeka Long Term Rentals Frontend - Complete Documentation

Table of Contents

Project Overview

Reeka Long Term Rentals Frontend is a modern, full-featured property management system built with Next.js 15. The application provides comprehensive tools for managing properties, tenants, maintenance requests, portfolios, and financial reporting for long-term rental properties.

Key Features

- **Property Management**: Create, edit, and manage property listings with detailed information
- **Tenant Management**: Track tenant information, lease agreements, and rental history
- **Maintenance Tracking**: Create and manage maintenance tickets with status tracking
- **Portfolio Management**: Organize properties into portfolios for better management
- **Financial Reporting**: Generate reports for expenses, revenue, and financial analytics
- **Role-Based Access Control**: Multi-level user roles with granular permissions
- **Dashboard Analytics**: Visual insights with charts and metrics
- **Responsive Design**: Mobile-first approach with Tailwind CSS

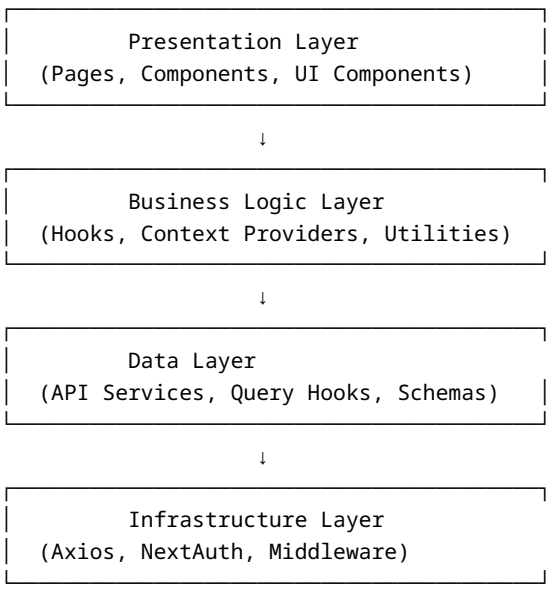
Architecture

High-Level Architecture

The application follows a **modern Next.js App Router architecture** with the following key principles:

1. **Server Components by Default**: Leverages React Server Components for optimal performance
2. **Client Components When Needed**: Uses `'use client'` directive for interactive components
3. **API Route Handlers**: Next.js API routes for authentication and server-side logic
4. **Middleware Protection**: Route-level authentication and authorization via Next.js middleware
5. **Service Layer Pattern**: Centralized API services with consistent error handling
6. **Query Layer**: TanStack Query for data fetching, caching, and state synchronization

Architecture Layers



Tech Stack

Core Framework

- **Next.js 15.2.5**: React framework with App Router
- **React 19.0.0**: UI library
- **TypeScript 5.x**: Type safety and developer experience

Styling

- **Tailwind CSS 4.1.3**: Utility-first CSS framework
- **tailwindcss-animate**: Animation utilities
- **next-themes**: Dark mode support
- **Custom Design System**: Radix UI components with custom styling

UI Components

- **Radix UI**: Accessible component primitives
  - `@radix-ui/react-checkbox`
  - `@radix-ui/react-popover`
  - `@radix-ui/react-select`
  - `@radix-ui/react-tooltip`
- **Ant Design 5.26.4**: Additional UI components
- **Lucide React**: Icon library

State Management & Data Fetching

- **TanStack Query (React Query) 5.72.2**: Server state management
- **React Context API**: Client-side state for UI state
- **Formik 2.4.6**: Form state management
- **Yup 1.6.1**: Schema validation

Authentication

- **NextAuth.js 4.24.11**: Authentication and session management
- **JWT Strategy**: Token-based authentication

HTTP Client

- **Axios 1.8.4**: HTTP client with interceptors

Date Handling

- **date-fns 4.1.0**: Date utility library
- **dayjs 1.11.13**: Lightweight date library
- **react-datepicker 8.3.0**: Date picker component
- **react-day-picker 9.6.7**: Calendar component

Data Visualization

- **Chart.js 4.4.9**: Charting library
- **react-chartjs-2 5.3.0**: React wrapper for Chart.js

Notifications

- **react-toastify 11.0.5**: Toast notifications

Development Tools

- **ESLint 9**: Code linting
- **Prettier 3.5.3**: Code formatting
- **Husky 8.0.0**: Git hooks
- **Commitizen**: Conventional commits
- **TypeScript**: Type checking

Testing

- **Playwright 1.51.1**: End-to-end testing
- **Vitest 3.1.1**: Unit testing framework

Build Tools

- **Turbopack**: Fast bundler (Next.js default)
- **PostCSS**: CSS processing

Project Structure

```
reeka_ltr_frontend/
├── .github/                # GitHub workflows and templates
├── .husky/                 # Git hooks configuration
├── playwright/             # E2E test files
├── public/                 # Static assets
├── src/
│   ├── app/                # Next.js App Router
│   │   ├── api/            # API route handlers
│   │   │   └── auth/       # NextAuth configuration
│   │   ├── auth/           # Authentication pages
│   │   │   ├── signin/     # Sign in page
│   │   │   ├── signup/     # Sign up page
│   │   │   ├── verify-email/# Email verification
│   │   │   ├── forgot-password/ # Password recovery
│   │   │   └── reset-password/  # Password reset
│   │   ├── constants/      # Application constants
│   │   │   ├── index.ts
│   │   │   ├── maintenance.ts
│   │   │   ├── property.ts
│   │   │   └── roles.ts    # Role definitions and permissions
│   │   ├── dashboard/      # Dashboard page
│   │   ├── listings/       # Property listings
│   │   │   ├── add-property/# Add new property
│   │   │   ├── portfolio/  # Portfolio view
│   │   │   └── property/   # Property detail view
│   │   ├── maintenance/    # Maintenance tickets
│   │   ├── reports/        # Financial reports
│   │   ├── settings/       # User and organization settings
│   │   ├── tenants/        # Tenant management
│   │   ├── error.tsx       # Error boundary
│   │   ├── layout.tsx      # Root layout
│   │   ├── page.tsx        # Home page
│   │   └── globals.css     # Global styles
│   ├── components/         # React components
```

```
│   │   ├── dashboard/       # Dashboard-specific components
│   │   │   ├── charts/      # Chart components
│   │   │   ├── overview.tsx
│   │   │   ├── portfolio.tsx
│   │   │   └── properties.tsx
│   │   ├── hocs/            # Higher-order components
│   │   │   └── with-role-protection.tsx
│   │   ├── listings/        # Property listing components
│   │   ├── maintenance/     # Maintenance components
│   │   ├── portfolio/       # Portfolio management components
│   │   ├── property/        # Property form components
│   │   ├── providers/       # Context providers
│   │   │   └── query-provider.tsx
│   │   ├── reports/         # Report components
│   │   ├── settings/        # Settings components
│   │   ├── tabs/            # Tab components
│   │   ├── tenants/         # Tenant components
│   │   └── ui/              # Reusable UI components
│   │       ├── button.tsx
│   │       ├── card.tsx
│   │       ├── modal.tsx
│   │       ├── sidebar/
│   │       └── ...
│   ├── hooks/               # Custom React hooks
│   │   ├── use-debounce.ts
│   │   ├── use-mobile.tsx
│   │   └── use-role-navigation.ts
│   ├── lib/                 # Utility functions
│   │   └── utils.ts
│   ├── middleware.ts        # Next.js middleware (auth & routing)
│   ├── services/            # Service layer
│   │   ├── api/             # API service functions
│   │   │   ├── api-service.ts  # Axios instance & interceptors
│   │   │   ├── auth.ts
│   │   │   ├── properties.ts
│   │   │   ├── tenants.ts
│   │   │   ├── maintenance.ts
│   │   │   ├── portfolios.ts
│   │   │   ├── dashboard.ts
│   │   │   ├── report.ts
│   │   │   └── schemas/     # TypeScript schemas
│   │   └── queries/         # TanStack Query hooks
│   │       ├── factories/   # Query factory functions
│   │       ├── hooks/       # Custom query hooks
│   │       └── utils/      # Query utilities
│   └── types/               # TypeScript type definitions
│       └── next-auth.d.ts   # NextAuth type extensions
├── .env                     # Environment variables (not in git)
├── .gitignore
├── commitlint.config.cjs    # Commit message linting
├── components.json          # shadcn/ui configuration
├── eslint.config.mjs        # ESLint configuration
├── next.config.ts           # Next.js configuration
├── package.json
```

```
├── playwright.config.ts     # Playwright configuration
├── postcss.config.mjs       # PostCSS configuration
├── prettier.config.js       # Prettier configuration
├── pnpm-lock.yaml           # Package lock file
├── tailwind.config.ts       # Tailwind CSS configuration
├── tsconfig.json            # TypeScript configuration
└── README.md                # Basic project documentation
```

Getting Started

Prerequisites

- **Node.js**: 22.12.0 or later
- **Package Manager**: npm, yarn, or pnpm (recommended)
- **Git**: For version control

Installation

1. **Clone the repository**:

```
git clone [repository-url]
cd reeka_ltr_frontend
```

2. **Install dependencies**:

```
npm install
# or
yarn install
# or
pnpm install
```

3. **Set up environment variables**:

   Create a `.env` file in the root directory:

```
# API Configuration
NEXT_PUBLIC_API_BASE_URL=http://localhost:8080/api/v1

# NextAuth Configuration
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-secret-key-here
```

   Note: Generate a secure `NEXTAUTH_SECRET` using:

```
openssl rand -base64 32
```

4. **Run the development server**:

```
npm run dev
# or
yarn dev
# or
pnpm dev
```

5. **Open your browser**: Navigate to http://localhost:3000

Available Scripts

Script	Description
`npm run dev`	Start development server with Turbopack
`npm run build`	Build production bundle
`npm run start`	Start production server
`npm run lint`	Run ESLint
`npm run lint:fix`	Fix ESLint errors automatically
`npm run type-check`	Run TypeScript type checking
`npm run format`	Format code with Prettier
`npm run test`	Run unit tests with Vitest
`npm run test:e2e`	Run end-to-end tests with Playwright
`npm run test:e2e:ui`	Run Playwright tests in UI mode
`npm run commit:fix`	Fix commit message format

Authentication & Authorization

Authentication Flow

The application uses **NextAuth.js** with a credentials provider for authentication.

Authentication Process

1. **User Sign In**: User enters email and password
2. **Credentials Validation**: NextAuth calls the `authorize` function
3. **API Authentication**: Backend API validates credentials
4. **Token Generation**: Backend returns access and refresh tokens
5. **Session Creation**: NextAuth creates a JWT session
6. **Role Validation**: Middleware validates user role against allowed roles

```
// src/app/api/auth/[...nextauth]/auth.ts
```

Authentication Configuration Key features: - **JWT Strategy**: Sessions stored as JWTs - **Token Management**: Access and refresh tokens stored in session - **Role Validation**: Only users with allowed roles can access the app - **Session Duration**: 24 hours (configurable)

Authorization System

The application implements a **role-based access control (RBAC)** system with two levels:

1. Module-Level Access Controls which major sections a user can access:

Role	Allowed Modules
Admin	dashboard, listings, tenants, maintenance, reports, settings
Property Manager	listings, tenants, maintenance, reports
Associate Manager	listings, tenants, maintenance, reports
Maintenance	maintenance, settings

2. Action-Level Permissions Controls specific actions within modules:

Role	Key Actions
Admin	Full CRUD on all resources, manage settings
Property Manager	Edit properties, manage tenants, view reports
Associate Manager	View properties, manage tenants, view reports
Maintenance	View and edit maintenance tickets only

Role Configuration Roles are defined in `src/app/constants/roles.ts`:

```
export const allowedRoles = [
  'Maintenance',
  'Property Manager',
  'Admin',
  'Associate Manager',
];

export const modulePermissions = {
  'Admin': ['dashboard', 'listings', 'tenants', 'maintenance', 'reports', 'settings'],
  // ... other roles
};
```

Route Protection

Middleware Protection The `src/middleware.ts` file protects routes at the edge:

1. **Authentication Check**: Verifies user has valid session
2. **Role Validation**: Ensures user has an allowed role
3. **Route Access Check**: Validates user can access specific route
4. **Automatic Redirects**: Redirects unauthorized users to appropriate pages

Protected Routes All routes except authentication pages are protected by default. The middleware: - Allows access to `/auth/*` routes - Allows access to error pages (`/forbidden`, `/not-found`, `/server-error`) - Validates role-based access for all other routes - Redirects root (`/`) to first allowed module based on role

Component-Level Protection Use the `withRoleProtection` HOC for component-level protection:

```
import { withRoleProtection } from '@/components/hocs/with-role-protection';

const ProtectedComponent = withRoleProtection(
  ['Admin', 'Property Manager'],
  MyComponent
);
```

API Integration

API Service Architecture

The application uses a **layered API architecture**:

1. **API Service Layer** (`src/services/api/`): Axios-based HTTP client
2. **Query Layer** (`src/services/queries/`): TanStack Query hooks
3. **Component Layer**: React components consuming query hooks

API Client Setup

The main API client is configured in `src/services/api/api-service.ts`:

```
// Base URL from environment variable
const baseURL = process.env.NEXT_PUBLIC_API_BASE_URL || 'http://localhost:8080/api/v1';

export const api = axios.create({
  baseURL,
  headers: {
    'Content-Type': 'application/json',
  },
});
```

Request Interceptor Automatically adds authentication token to requests:

```
api.interceptors.request.use(async config => {
  const session = await getSession();
  if (session?.accessToken) {
    config.headers.Authorization = `Bearer ${session.accessToken}`;
  }
  return config;
});
```

Response Interceptor Handles authentication errors:

```
api.interceptors.response.use(
  response => response,
  async (error: AxiosError) => {
    if (error.response?.status === 401) {
      await signOut({ redirect: true, callbackUrl: '/auth/signin' });
    }
    return Promise.reject(error);
  }
);
```

API Service Modules

The API is organized into service modules:

- **auth.ts**: Authentication endpoints
- **properties.ts**: Property management
- **tenants.ts**: Tenant management
- **maintenance.ts**: Maintenance tickets
- **portfolios.ts**: Portfolio management
- **dashboard.ts**: Dashboard data
- **report.ts**: Report generation
- **users.ts**: User management
- **organizations.ts**: Organization management
- **countries.ts**: Country data
- **upload.ts**: File uploads

Query Layer

The query layer provides a consistent interface for data fetching using TanStack Query.

Query Factory Pattern The application uses factory functions to create consistent query hooks:

```
// src/services/queries/factories/
export function useQueryFactory<TData, TParams>(config) {
  // Standardized query configuration
}
```

Custom Query Hooks Each domain has custom hooks in `src/services/queries/hooks/`:

- `useAuth.ts`: Authentication queries
- `useProperties.ts`: Property queries
- `useTenants.ts`: Tenant queries
- `useMaintenance.ts`: Maintenance queries
- `usePortfolios.ts`: Portfolio queries
- `useDashboard.ts`: Dashboard queries
- `useReport.ts`: Report queries
- `useUser.ts`: User queries

```
import { useProperties } from '@/services/queries/hooks/useProperties';

function PropertiesList() {
  const { data, isLoading, isError, paginationMeta } = useProperties({
    search: 'apartment',
    status: 'listed',
  });

  if (isLoading) return <div>Loading...</div>;
  if (isError) return <div>Error occurred</div>;

  return (
    <div>
      {data?.map(property => (
        <PropertyCard key={property.id} property={property} />
      ))}
      {paginationMeta && (
        <Pagination meta={paginationMeta} />
      )}
    </div>
  );
}
```

Using Query Hooks

Error Handling The query layer automatically handles HTTP errors:

- **401 Unauthorized**: Redirects to `/auth/signin`
- **403 Forbidden**: Redirects to `/forbidden`
- **404 Not Found**: Redirects to `/not-found`
- **500 Server Error**: Redirects to `/server-error`
- **Other Errors**: Redirects to `/error`

State Management

Server State (TanStack Query)

TanStack Query handles all server state:

- **Automatic Caching**: Queries are cached automatically
- **Background Refetching**: Data stays fresh
- **Optimistic Updates**: UI updates before server confirmation
- **Pagination Support**: Built-in pagination handling
- **Error Handling**: Centralized error management

Client State

React Context API is used for client-side UI state:

- **Theme Context**: Dark/light mode (via `next-themes`)
- **Form Context**: Form state management (via Formik)
- **Portfolio Context**: Portfolio management state
- **Property Form Context**: Property form state

Session State

NextAuth.js manages authentication state:

- **Session Provider**: Wraps the application
- **Session Hooks**: `useSession()` for accessing session data
- **Automatic Refresh**: Session tokens refreshed automatically

---

Component Architecture

Component Organization

Components are organized by feature/domain:

```
components/
├── dashboard/      # Dashboard-specific components
├── listings/       # Property listing components
├── maintenance/    # Maintenance components
├── portfolio/      # Portfolio components
├── property/       # Property form components
├── reports/        # Report components
├── settings/       # Settings components
├── tenants/        # Tenant components
└── ui/             # Reusable UI primitives
```

Component Patterns

```tsx
// app/dashboard/page.tsx
export default async function DashboardPage() {
  // Server component - can fetch data directly
  const data = await fetchDashboardData();
  return <DashboardClient data={data} />;
}
```

1. Server Components (Default)

```tsx
// components/dashboard/DashboardClient.tsx
'use client';

import { useDashboard } from '@/services/queries/hooks/useDashboard';

export function DashboardClient() {
  const { data } = useDashboard();
  // Client-side interactivity
  return <div>{/* Interactive UI */}</div>;
}
```

2. Client Components

```tsx
// UI components that work together
<Modal>
  <ModalHeader>Title</ModalHeader>
  <ModalBody>Content</ModalBody>
  <ModalFooter>Actions</ModalFooter>
</Modal>
```

3. Compound Components

UI Component Library

The application uses **Radix UI** primitives with custom styling:

- **Accessible**: Built on Radix UI for accessibility
- **Customizable**: Styled with Tailwind CSS
- **Type-Safe**: Full TypeScript support
- **Consistent**: Design system approach

Key UI components: - `Button`: Primary, secondary, destructive variants - `Card`: Container component - `Modal`: Dialog/modal component - `Dropdown`: Dropdown menu - `DateRangePicker`: Date selection - `Pagination`: Pagination controls - `Sidebar`: Navigation sidebar - `Badge`: Status indicators - `Tooltip`: Tooltip component

---

Styling & Theming

Tailwind CSS Configuration

The application uses **Tailwind CSS 4.1.3** with a custom configuration:

```tsx
// tailwind.config.ts
colors: {
  primary: {
    DEFAULT: '#e36b37', // Orange primary color
    foreground: 'hsl(var(--primary-foreground))',
  },
  // ... other colors
}
```

Color System

Typography Custom fonts loaded via Next.js font optimization:

- **Nunito**: Primary font (weights: 400, 500, 600, 700, 800)
- **Inter**: Sans-serif font
- **Modak**: Display font

Responsive Design Mobile-first approach with breakpoints: - `sm`: 640px - `md`: 768px - `lg`: 1024px - `xl`: 1280px - `2xl`: 1400px

Dark Mode

Dark mode support via `next-themes`:

```
import { useTheme } from 'next-themes';

function ThemeToggle() {
  const { theme, setTheme } = useTheme();
  return (
    <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      Toggle theme
    </button>
  );
}
```

CSS Variables

Design tokens defined as CSS variables in `globals.css`:

```
:root {
  --background: 0 0% 100%;
  --foreground: 222.2 84% 4.9%;
  --primary: 14 76% 50%;
  /* ... more variables */
}
```

Routing & Navigation

App Router Structure

The application uses **Next.js App Router** with file-based routing:

```
app/
├── page.tsx              # / (root)
├── dashboard/
│   └── page.tsx          # /dashboard
├── listings/
│   ├── page.tsx          # /listings
│   ├── add-property/
│   │   └── page.tsx      # /listings/add-property
│   └── property/
│       └── page.tsx      # /listings/property
└── ...
```

Route Protection

Routes are protected via middleware:

1. **Authentication**: Must be logged in
2. **Role Validation**: Must have allowed role
3. **Route Access**: Must have permission for specific route

Navigation

Navigation is role-aware:

```
// hooks/use-role-navigation.ts
export function useRoleNavigation() {
  const { data: session } = useSession();
  const role = session?.user?.role;
  const allowedModules = getAllowedModules(role);
  // Returns navigation items based on role
}
```

Dynamic Routes

Dynamic routes are supported:

```
// app/listings/property/[id]/page.tsx
export default function PropertyPage({ params }: { params: { id: string } }) {
  // Access params.id
}
```

Form Handling

Form Libraries

The application uses **Formik** for form state and **Yup** for validation:

```
import { Formik, Form, Field } from 'formik';
import * as Yup from 'yup';

const validationSchema = Yup.object({
  email: Yup.string().email('Invalid email').required('Required'),
  password: Yup.string().min(8, 'Too short').required('Required'),
});

function LoginForm() {
  return (
    <Formik
      initialValues={{ email: '', password: '' }}
      validationSchema={validationSchema}
      onSubmit={async (values) => {
        await signIn(values);
      }}
    >
      <Form>
        <Field name="email" type="email" />
        <Field name="password" type="password" />
        <button type="submit">Submit</button>
      </Form>
    </Formik>
```

```
  );
}
```

Form Context Pattern

Complex forms use context for state management:

```
// components/property/property-form-context.tsx
export function PropertyFormProvider({ children }) {
  const [formState, setFormState] = useState(initialState);
  // Form state management
  return (
    <PropertyFormContext.Provider value={{ formState, setFormState }}>
      {children}
    </PropertyFormContext.Provider>
  );
}
```

Testing

Testing Strategy

The application uses a multi-layered testing approach:

1. **Unit Tests**: Vitest for component and utility testing
2. **E2E Tests**: Playwright for end-to-end testing

Playwright Configuration

E2E tests are configured in `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './playwright',
  use: {
    baseURL: 'http://localhost:3000',
  },
});
```

Running Tests

```
# Unit tests
npm run test

# E2E tests
npm run test:e2e

# E2E tests with UI
npm run test:e2e:ui
```

Test Examples

```
// playwright/example.spec.ts
import { test, expect } from '@playwright/test';

test('should load dashboard', async ({ page }) => {
  await page.goto('/dashboard');
  await expect(page).toHaveTitle(/Reeka/);
});
```

Code Quality

Linting

ESLint is configured with multiple plugins:

- `eslint-config-next`: Next.js specific rules
- `eslint-plugin-import`: Import/export rules
- `eslint-plugin-playwright`: Playwright test rules
- `eslint-plugin-prettier`: Prettier integration
- `eslint-plugin-unicorn`: Best practices
- `eslint-plugin-simple-import-sort`: Import sorting

Formatting

Prettier handles code formatting:

```
npm run format
```

Configuration in `prettier.config.js` with Tailwind CSS plugin for class sorting.

Type Checking

TypeScript provides compile-time type checking:

```
npm run type-check
```

Git Hooks

Husky manages git hooks:

- **Pre-commit**: Runs linting and type checking
- **Commit-msg**: Validates commit message format (Commitizen)

Commit Convention

The project uses **Conventional Commits**:

```
feat: add property search functionality
fix: resolve authentication token refresh issue
docs: update API documentation
style: format code with Prettier
refactor: reorganize component structure
test: add unit tests for utility functions
chore: update dependencies
```

Deployment

Build Process

1. **Install Dependencies**:
   ```
   npm install
   ```
2. **Build Application**:
   ```
   npm run build
   ```
3. **Start Production Server**:
   ```
   npm run start
   ```

Environment Variables

Ensure all environment variables are set in production:

```
NEXT_PUBLIC_API_BASE_URL=https://api.example.com/api/v1
NEXTAUTH_URL=https://app.example.com
NEXTAUTH_SECRET=production-secret-key
```

Deployment Platforms

The application can be deployed to:

- **Vercel** (Recommended for Next.js)
- **Netlify**
- **AWS Amplify**
- **Docker** containers
- **Traditional hosting** with Node.js support

Docker Deployment

Example Dockerfile:

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

Troubleshooting

Common Issues

1. Authentication Not Working **Problem**: Users cannot sign in

Solutions: - Verify `NEXTAUTH_SECRET` is set - Check `NEXTAUTH_URL` matches your domain - Ensure API base URL is correct - Check browser console for errors

2. API Requests Failing **Problem**: API calls return 401 or 403

Solutions: - Verify access token is being sent (check Network tab) - Check token expiration - Verify API base URL is correct - Check CORS settings on backend

3. Build Errors **Problem**: `npm run build` fails

Solutions: - Run `npm run type-check` to find TypeScript errors - Run `npm run lint` to find linting errors - Clear `.next` directory and rebuild - Check for missing environment variables

4. Styling Issues **Problem**: Styles not applying correctly

Solutions: - Verify Tailwind CSS is configured correctly - Check `tailwind.config.ts` content paths - Ensure CSS imports are correct - Clear browser cache

5. Role-Based Access Issues **Problem**: Users cannot access certain routes

Solutions: - Verify user role in session - Check `src/app/constants/roles.ts` configuration - Review middleware logic - Check route-to-module mapping

Debugging Tips

1. **Enable Debug Logging**:

```
// Add to api-service.ts
console.log('[API] Request:', config.method, config.url);
```

2. **Check Session Data**:

```
const { data: session } = useSession();
console.log('Session:', session);
```

3. **Inspect Query State**:

```
const query = useProperties();
console.log('Query state:', query);
```

4. **Network Tab**: Use browser DevTools to inspect API requests

---

Additional Resources

Documentation Links

- Next.js Documentation
- TanStack Query Documentation
- NextAuth.js Documentation
- Tailwind CSS Documentation
- Radix UI Documentation

Project-Specific Documentation

- `src/services/queries/README.md`: Query layer documentation
- Component README files (if present)

Support

For issues or questions: 1. Check existing documentation 2. Review code comments 3. Check GitHub issues 4. Contact the development team

---

Contributing

Development Workflow

1. **Create Feature Branch**:
   ```
   git checkout -b feature/your-feature-name
   ```
2. **Make Changes**: Follow coding standards and conventions
3. **Run Tests**:
   ```
   npm run test
   npm run test:e2e
   ```
4. **Lint and Format**:
   ```
   npm run lint:fix
   npm run format
   ```
5. **Commit Changes**:
   ```
   npm run commit:fix
   ```
6. **Push and Create PR**: Push to remote and create pull request

Code Standards

- **TypeScript**: Use TypeScript for all new code
- **Components**: Use functional components with hooks
- **Naming**: Use PascalCase for components, camelCase for functions
- **Imports**: Sort imports automatically (handled by ESLint)
- **Comments**: Add comments for complex logic
- **Documentation**: Update documentation for new features

License

MIT License

Last Updated: 2024

Version: 0.1.0